Documento de apoyo (Sesión 9, Unidad 2)

Gobierno de Datos y Machine Learning en Entornos Cloud con BigQuery

Arquitectura, control de acceso, optimización analítica y modelos predictivos con SQL

Javier Andrés Valdez González / Auxiliar

Seminario de Sistemas 2 — Sección P

Universidad de San Carlos de Guatemala

Abril de 2026

Resumen

BigQuery se ha consolidado como una plataforma analítica *serverless* de referencia para construir soluciones de datos escalables en la nube. Su propuesta combina almacenamiento columnar administrado, motor de consulta distribuido, seguridad integrada y capacidades de aprendizaje automático ejecutadas directamente sobre SQL mediante BigQuery ML. Este documento desarrolla, con enfoque técnico-académico, los fundamentos de BigQuery, su arquitectura, sus esquemas de precios y permisos, y la forma en que los principios de gobierno de datos se aplican en escenarios reales de analítica empresarial. Asimismo, se abordan estrategias de optimización de consultas, uso de vistas y tablas materializadas, creación de modelos predictivos con `CREATE MODEL`, y exportación de resultados hacia herramientas externas o flujos complementarios de ciencia de datos.

Palabras clave: BigQuery, gobierno de datos, BigQuery ML, Google Cloud, IAM, vistas materializadas, optimización SQL, analítica en la nube.

Índice

1. Introducción **4**

2. Fundamentos de BigQuery **5**
2.1. ¿Qué es BigQuery? 5
2.2. Arquitectura conceptual 5
2.3. BigQuery dentro de una arquitectura moderna de datos 6

3. Gobierno de datos con BigQuery **7**
3.1. Concepto de gobierno de datos 7
3.2. Recursos y niveles de control 7
3.3. IAM, roles y permisos 8
3.4. Seguridad a nivel de columna, fila y vista 8

4. Arquitectura, precios y permisos **9**
4.1. Modelo de precios 9
4.2. On-demand vs. capacidad reservada 9
4.3. Permisos como parte de la arquitectura 9

5. Aplicación del gobierno de datos en BigQuery **11**
5.1. Diseño por dominios y sensibilidad 11
5.2. Principio de mínimo privilegio 11
5.3. Compartición controlada con vistas 11
5.4. Trazabilidad y auditoría 12

6. Optimización de consultas en BigQuery **13**
6.1. Principio general: consultar menos datos 13
6.2. Buenas prácticas fundamentales 13
6.3. Ejemplo de optimización 13
6.4. Materialización de transformaciones 14

7. Tablas materializadas y vistas **15**
7.1. Vistas lógicas 15
7.2. Vistas materializadas 15
7.3. Authorized materialized views 15
7.4. Ejemplo de vista materializada 15

8. BigQuery ML **17**
8.1. ¿Qué es BigQuery ML? 17
8.2. Flujo general de trabajo 17
8.3. Ejemplo básico de clasificación 17
8.4. Interpretación crítica de resultados 18

9. Exportación a herramientas externas **20**
9.1. Exportación de datos 20
9.2. Exportación de modelos 20
9.3. Herramientas de consumo habituales 20

10. Caso práctico integrador **22**
10.1. Diseño propuesto 22
10.2. Consulta agregada de soporte al modelo 22

11. Conclusiones **24**

1. Introducción

La adopción de plataformas de datos en la nube ha transformado la manera en que las organizaciones diseñan sus procesos analíticos. En lugar de depender exclusivamente de servidores locales, capacidad fija y ciclos largos de aprovisionamiento, los equipos modernos operan con servicios administrados que escalan bajo demanda y permiten integrar almacenamiento, consulta, seguridad y modelado avanzado en una misma plataforma. En este contexto, BigQuery ocupa un lugar central dentro del ecosistema de Google Cloud al ofrecer un almacén analítico totalmente administrado, orientado a consultas masivas y a la construcción de productos de datos listos para análisis y decisión [8], [17].

Sin embargo, la disponibilidad tecnológica no garantiza por sí sola un uso correcto de los datos. A medida que crece el volumen de información, también crece la necesidad de gobernarla: definir quién accede, bajo qué condiciones, con qué trazabilidad y con qué nivel de protección para columnas o registros sensibles. El gobierno de datos en BigQuery no debe entenderse como una capa separada del análisis, sino como un conjunto de prácticas y controles integrados en el diseño del *dataset*, los permisos, las vistas, las etiquetas de política y los mecanismos de auditoría [6], [13].

La sesión 9 articula estas dos dimensiones. Por un lado, estudia cómo BigQuery soporta análisis eficientes mediante optimización de consultas, uso de vistas materializadas y separación entre cómputo y almacenamiento. Por otro, incorpora BigQuery ML como extensión natural del análisis, permitiendo crear modelos de clasificación, regresión, segmentación o pronóstico directamente con GoogleSQL, sin salir del entorno analítico [7], [19]. El propósito de este documento es desarrollar esos fundamentos conceptuales y prácticos para que el estudiante comprenda no solo cómo consultar datos en la nube, sino cómo gobernarlos, modelarlos y convertirlos en insumos confiables para decisiones organizacionales.

2. Fundamentos de BigQuery

2.1. ¿Qué es BigQuery?

BigQuery es una plataforma analítica *serverless* de Google Cloud diseñada para almacenar y consultar grandes volúmenes de datos mediante SQL estándar de Google (*GoogleSQL*) [8]. Su valor principal radica en abstraer la administración de infraestructura: el usuario no configura servidores, discos, parches ni clústeres, sino que se concentra en cargar datos, definir estructuras y ejecutar cargas o consultas analíticas.

Desde el punto de vista funcional, BigQuery combina varias capacidades que antes solían distribuirse en herramientas separadas:

- **Almacenamiento analítico columnar** para tablas, vistas materializadas, clones y *snapshots*.
- **Motor de ejecución distribuido** capaz de paralelizar operaciones sobre grandes volúmenes.
- **Seguridad y control de acceso** a nivel de proyecto, *dataset*, tabla, columna, fila y vista.
- **Capacidades analíticas avanzadas** como funciones geoespaciales, búsqueda, BI Engine y BigQuery ML.

2.2. Arquitectura conceptual

Uno de los rasgos arquitectónicos más importantes de BigQuery es la separación entre almacenamiento y cómputo. Google documenta que ambos componentes escalan de forma independiente, lo que reduce contención entre carga, lectura y administración [17]. De forma simplificada, la arquitectura puede entenderse así:

A1. Capa de almacenamiento: conserva tablas en formato optimizado para análisis, con replicación y cifrado administrado.

A2. Capa de cómputo: distribuye la ejecución de consultas en paralelo mediante *workers* y *slots*.

A3. Capa de metadatos y gobierno: administra esquemas, permisos, etiquetas, linaje y catálogos.

A4. Capa de consumo: expone resultados a SQL, APIs, conectores, notebooks y herramientas de visualización.

Esta separación es especialmente relevante en contextos educativos porque ayuda a distinguir el almacenamiento persistente del costo de procesamiento. En un sistema relacional tradicional suele pensarse en un servidor único; en BigQuery, en cambio, el análisis se paga y administra como capacidad consumida por las consultas, mientras el almacenamiento se factura de forma separada [9], [21].

2.3. BigQuery dentro de una arquitectura moderna de datos

Dentro de una arquitectura analítica, BigQuery puede recibir datos desde archivos, procesos batch, APIs, servicios de transferencia, *streaming* y otras bases de datos. Una vez cargados, estos datos pueden organizarse en modelos tipo *staging*, *core*, *mart* o dominio analítico, y luego ser consumidos por analistas, científicos de datos o tableros empresariales.

F1. Ingesta: carga desde Cloud Storage, Data Transfer Service, Storage Write API u otras fuentes.

F2. Transformación: limpieza y modelado con SQL, procedimientos, tablas intermedias o ELT.

F3. Gobierno y publicación: definición de permisos, vistas autorizadas y políticas de acceso.

F4. Consumo y modelado: consultas analíticas, tableros, exportaciones y modelos con BigQuery ML.

BigQuery, por tanto, no es solo un repositorio. Funciona como motor de análisis, como espacio de gobierno y como plataforma para ejecutar ciertas tareas de aprendizaje automático sin requerir extracción previa hacia otro entorno.

3. Gobierno de datos con BigQuery

3.1. Concepto de gobierno de datos

El gobierno de datos puede definirse como el conjunto de políticas, responsabilidades y controles que aseguran que los datos se administren de manera consistente, segura y útil para la organización. En BigQuery, dicho gobierno se implementa mediante una combinación de metadatos, permisos, clasificación, trazabilidad y patrones de compartición de datos [13].

Desde una perspectiva práctica, gobernar datos en BigQuery implica responder preguntas como las siguientes:

- ¿Qué *dataset* contiene información sensible?
- ¿Qué usuarios solo pueden consultar datos agregados y no datos detallados?
- ¿Qué columnas deben restringirse por contener PII, montos salariales o identificadores internos?
- ¿Qué registros deben filtrarse según región, cliente o unidad de negocio?
- ¿Cómo se auditan los cambios en permisos y políticas?

3.2. Recursos y niveles de control

BigQuery aplica controles de acceso sobre distintos niveles de recurso. Esta granularidad es importante porque el principio de mínimo privilegio rara vez se satisface otorgando permisos amplios a todo el proyecto [4], [6].

Nivel	**Aplicación del control**
Proyecto	Habilita capacidades globales como ejecución de *jobs*, administración y visibilidad general.
Dataset	Controla lectura, escritura y propiedad sobre conjuntos de tablas, vistas y rutinas.
Tabla o vista	Permite restringir acceso a objetos concretos sin exponer todo el *dataset*.
Columna	Se protege con *policy tags* para seguridad a nivel de columna.
Fila	Se restringe con *row-level access policies*.

3.3. IAM, roles y permisos

Google recomienda privilegiar roles predefinidos de IAM en lugar de roles básicos amplios, ya que estos últimos pueden conceder permisos excesivos [4]. En el trabajo cotidiano con BigQuery suelen intervenir, entre otros, los siguientes roles:

- **`roles/bigquery.user`:** permite crear *jobs* y trabajar con consultas en el proyecto.
- **`roles/bigquery.dataViewer`:** lectura de datos y metadatos.
- **`roles/bigquery.dataEditor`:** modificación de datos dentro de un *dataset*.
- **`roles/bigquery.dataOwner`:** control amplio sobre objetos y acceso dentro del *dataset*.
- **`roles/bigquery.admin`:** administración completa del servicio, generalmente a nivel de proyecto.

3.4. Seguridad a nivel de columna, fila y vista

BigQuery ofrece mecanismos diferenciados para controles finos:

G1. Column-level security: usa *policy tags* para clasificar columnas y limitar su lectura según permisos [5], [18].

G2. Row-level security: filtra registros según políticas asociadas a usuarios o grupos [12].

G3. Authorized views: comparten subconjuntos de datos sin dar acceso directo a las tablas fuente [3].

G4. Authorized datasets: autorizan conjuntos completos de vistas para acceder a datos de otro *dataset*, simplificando arquitecturas multiusuario [2].

El valor pedagógico de estos mecanismos es que permiten mostrar que la seguridad no es binaria. Un usuario puede ver un agregado por región sin ver el salario individual, o puede acceder únicamente a clientes de su país sin conocer el universo completo de registros.

4. Arquitectura, precios y permisos

4.1. Modelo de precios

BigQuery separa el costo de almacenamiento del costo de cómputo. Esta separación es coherente con su arquitectura y obliga a entender el análisis como una actividad medible en términos de datos procesados o capacidad reservada [9], [17].

De forma general, el precio se organiza en:

- **Almacenamiento:** bytes guardados en tablas, vistas materializadas, clones o *snapshots*.
- **Cómputo por demanda:** cobro según bytes procesados por consultas.
- **Cómputo por capacidad:** uso de reservas y *slots* dentro de ediciones de BigQuery.
- **Operaciones adicionales:** algunos servicios asociados, *streaming*, BI Engine y ciertas tareas de BigQuery ML.

4.2. On-demand vs. capacidad reservada

En el esquema *on-demand*, el costo depende de los datos leídos o procesados por la consulta. Esto resulta conveniente para laboratorios, pruebas y cargas analíticas intermitentes. En cambio, cuando la organización necesita costos más predecibles o alta concurrencia, puede adoptar reservas de capacidad (*slots*) bajo ediciones como Standard, Enterprise o Enterprise Plus [9], [21].

Desde la perspectiva docente, esta diferencia es útil para discutir dos ideas:

P1. una mala consulta cuesta más en modo *on-demand* porque procesa más datos;

P2. una mala consulta también es problemática en modo capacidad porque consume recursos compartidos y puede afectar concurrencia.

4.3. Permisos como parte de la arquitectura

En BigQuery, la arquitectura técnica no puede separarse del diseño de permisos. La manera en que se distribuyen *datasets*, tablas y vistas impacta directamente la administración del acceso. Por ejemplo, ubicar tablas sensibles y vistas públicas en el mismo *dataset* suele dificultar la aplicación de privilegios mínimos. Por ello, una práctica frecuente es separar:

- ***datasets* crudos o restringidos,** con acceso muy limitado;

- ***datasets* curados o modelados,** donde trabajan procesos ETL/ELT;
- ***datasets* de consumo,** con vistas, agregados o modelos listos para usuarios finales.

La segmentación por dominio, entorno o sensibilidad simplifica la gobernanza y reduce riesgos de exposición accidental.

5. Aplicación del gobierno de datos en BigQuery

5.1. Diseño por dominios y sensibilidad

Aplicar gobierno de datos implica traducir principios abstractos en decisiones concretas de modelado y operación. Una primera práctica consiste en clasificar los activos por sensibilidad y por dominio de negocio. Por ejemplo, ventas, recursos humanos y finanzas pueden estar en *datasets* distintos, no solo por orden lógico, sino porque sus reglas de acceso son diferentes.

En un proyecto académico o empresarial se recomienda:

D1. identificar tablas con datos públicos, internos, confidenciales o sensibles;

D2. separar *datasets* según entorno: *dev*, *test*, *prod*;

D3. aplicar *policy tags* a columnas de identificación personal o financiera;

D4. publicar vistas para consumo cuando los usuarios no necesitan ver la tabla completa.

5.2. Principio de mínimo privilegio

El principio de mínimo privilegio implica conceder solo los permisos estrictamente necesarios para una función concreta. En BigQuery, esto se traduce en evitar el uso indiscriminado de `Owner`, `Editor` o `bigquery.admin` cuando bastan roles más acotados [4], [6].

Un patrón razonable podría ser:

- analistas con acceso de lectura a vistas o tablas curadas;
- ingenieros de datos con permisos de escritura en zonas de transformación;
- administradores con capacidad para crear políticas, taxonomías y monitorear auditoría;
- cuentas de servicio con permisos delimitados al *pipeline* que ejecutan.

5.3. Compartición controlada con vistas

Las vistas autorizadas son particularmente valiosas en entornos educativos porque muestran cómo compartir datos sin duplicarlos. En vez de crear múltiples tablas derivadas para cada audiencia, puede definirse una vista que excluya columnas sensibles o que filtre registros por regla de negocio [3].

Por ejemplo, una universidad podría exponer una vista con métricas agregadas de rendimiento académico para coordinación, mientras conserva los expedientes detallados en una tabla restringida. Esta aproximación favorece consistencia, reduce redundancia y centraliza las reglas de exposición.

5.4. Trazabilidad y auditoría

Un gobierno serio no termina al definir permisos. También requiere rastrear quién cambió una política, quién ejecutó consultas sensibles o qué procesos modificaron datos. BigQuery se integra con capacidades de auditoría y con el ecosistema de monitoreo de Google Cloud, lo que permite analizar eventos relacionados con acceso, *jobs* y políticas sobre etiquetas [1].

En un laboratorio, esta dimensión puede discutirse como parte de la responsabilidad profesional: un modelo predictivo o un *dashboard* puede influir en decisiones reales, por lo que su origen, permisos y trazabilidad deben quedar documentados.

6. Optimización de consultas en BigQuery

6.1. Principio general: consultar menos datos

Google resume la optimización de BigQuery con una idea central: las consultas que hacen menos trabajo suelen ser más rápidas y más baratas [15], [16]. En consecuencia, optimizar no significa únicamente escribir SQL "correcto", sino reducir bytes leídos, operaciones de mezcla, materialización intermedia y resultados innecesarios.

6.2. Buenas prácticas fundamentales

Entre las prácticas más importantes se encuentran:

- evitar `SELECT *` cuando no se necesitan todas las columnas;
- filtrar temprano con `WHERE`;
- usar particionamiento temporal o por rangos en lugar de fragmentar tablas por fecha;
- evitar *oversharding*;
- preagregar antes de ciertos `JOIN`;
- materializar transformaciones costosas cuando se reutilizan muchas veces;
- revisar el plan de ejecución y los detalles del *job*.

La optimización en BigQuery tiene un impacto dual: reduce latencia y también reduce costo en modelos *on-demand*. Por ello, el rigor técnico pedido en la sesión se vincula directamente con el diseño de consultas eficientes.

6.3. Ejemplo de optimización

Considérese una tabla de eventos particionada por fecha. Una consulta poco rigurosa podría leer datos de todo el historial:

Listing 1: Consulta poco eficiente

```
SELECT user_id, event_type, event_timestamp
FROM `proyecto.analytics.eventos`
WHERE event_type = 'purchase';
```

Si el análisis corresponde únicamente al mes de marzo de 2026, conviene restringir la partición y proyectar solo las columnas necesarias:

Listing 2: Consulta optimizada con filtro y proyección

```
SELECT user_id, event_timestamp
FROM `proyecto.analytics.eventos`
WHERE event_type = 'purchase'
  AND DATE(event_timestamp) BETWEEN '2026-03-01' AND '2026-03-31';
```

6.4. Materialización de transformaciones

La documentación de BigQuery advierte que algunas expresiones, CTE o transformaciones complejas pueden reevaluarse varias veces dentro de una consulta [16]. En escenarios ETL o analíticos recurrentes, puede ser más adecuado persistir resultados intermedios en tablas destino, tablas temporales o vistas materializadas.

Este criterio es importante cuando se aplican expresiones regulares, limpiezas repetidas, `JOIN` costosos o cálculos agregados que alimentan múltiples reportes.

7. Tablas materializadas y vistas

7.1. Vistas lógicas

Una vista lógica en BigQuery es una consulta almacenada que se ejecuta sobre tablas base cuando se consulta. Su ventaja principal es encapsular lógica analítica, exponer nombres de negocio y simplificar el acceso para usuarios finales. También puede utilizarse para ocultar columnas, aplicar filtros y presentar una capa semántica controlada.

7.2. Vistas materializadas

Las vistas materializadas son resultados precomputados que BigQuery mantiene y actualiza en segundo plano, reduciendo el costo de recalcular consultas repetitivas [14]. Pueden ser consultadas directamente o utilizadas por el optimizador para acelerar consultas sobre tablas base.

Su utilidad es alta cuando:

- existen agregaciones recurrentes sobre grandes volúmenes;
- las consultas se repiten con frecuencia;
- se necesita reducir tiempo de respuesta en tableros o reportes;
- el costo de refresco es menor que el costo acumulado de recalcular.

7.3. Authorized materialized views

BigQuery también permite vistas materializadas autorizadas, con lo cual se combina rendimiento con compartición segura de subconjuntos de información [3], [14]. Esto resulta útil cuando un equipo requiere acceso rápido a indicadores agregados, pero no debe ver el detalle completo de la fuente.

7.4. Ejemplo de vista materializada

Listing 3: Vista materializada para ventas diarias

```
CREATE MATERIALIZED VIEW `proyecto.mart.mv_ventas_diarias` AS
SELECT
  DATE(fecha_venta) AS fecha,
  region,
  SUM(monto) AS ventas_totales,
```

```
6   COUNT(*) AS transacciones
7 FROM `proyecto.core.ventas`
8 GROUP BY fecha, region;
```

Este patrón es adecuado para paneles ejecutivos que consultan permanentemente ventas por fecha y región. En lugar de recalcular la suma completa cada vez, BigQuery mantiene el resultado precomputado sujeto a las restricciones propias de las vistas materializadas.

8. BigQuery ML

8.1. ¿Qué es BigQuery ML?

BigQuery ML extiende GoogleSQL con sentencias para crear, entrenar, evaluar y usar modelos de aprendizaje automático directamente dentro de BigQuery [7], [19]. Su aporte pedagógico es significativo porque acerca el modelado predictivo a estudiantes que ya dominan SQL, sin exigirles migrar inmediatamente a otro ecosistema de programación.

Con BigQuery ML es posible entrenar, entre otros, modelos de:

- regresión lineal;
- regresión logística;
- *k-means* para segmentación;
- pronóstico de series de tiempo;
- bosques aleatorios y árboles potenciados en ciertos escenarios.

8.2. Flujo general de trabajo

Un flujo mínimo con BigQuery ML suele incluir:

M1. preparar una tabla o consulta de entrenamiento;

M2. crear el modelo con `CREATE MODEL`;

M3. revisar métricas con `ML.EVALUATE`;

M4. generar predicciones con `ML.PREDICT`;

M5. documentar supuestos, limitaciones y audiencia del modelo.

8.3. Ejemplo básico de clasificación

Supóngase un escenario donde se quiere predecir abandono de clientes a partir de variables de uso y facturación:

Listing 4: Creación de un modelo de regresión logística

```
CREATE OR REPLACE MODEL `proyecto.ml.modelo_churn`
OPTIONS(
  model_type='LOGISTIC_REG',
```

```
  input_label_cols=['abandono']
) AS
SELECT
  antiguedad_meses,
  consumo_promedio,
  incidencias_soporte,
  monto_mensual,
  abandono
FROM `proyecto.mart.clientes_entrenamiento`;
```

Una vez creado el modelo, puede evaluarse así:

Listing 5: Evaluación del modelo

```
SELECT *
FROM ML.EVALUATE(MODEL `proyecto.ml.modelo_churn`);
```

Y luego aplicarse a nuevos registros:

Listing 6: Predicción sobre nuevos clientes

```
SELECT *
FROM ML.PREDICT(
  MODEL `proyecto.ml.modelo_churn`,
  (
    SELECT
      antiguedad_meses,
      consumo_promedio,
      incidencias_soporte,
      monto_mensual
    FROM `proyecto.mart.clientes_scoring`
  )
);
```

8.4. Interpretación crítica de resultados

El uso de BigQuery ML no exime al analista de reflexionar sobre calidad de datos, sesgos, clases desbalanceadas, fuga de información o pertinencia del modelo. Pensamiento crítico, en esta sesión, significa no reducir el aprendizaje automático a ejecutar una sentencia exitosa, sino evaluar si las variables explican realmente el fenómeno y si las predicciones son apropiadas para el problema.

Un buen cierre analítico debería responder:

- qué variable objetivo se modeló;
- con qué datos y periodo se entrenó;
- qué métricas de evaluación se revisaron;
- qué limitaciones tiene el modelo;
- qué riesgo existe al usar sus resultados para decisiones reales o simuladas.

9. Exportación a herramientas externas

9.1. Exportación de datos

Aunque BigQuery permite cubrir una gran parte del ciclo analítico, es común que ciertos resultados se consuman fuera de la plataforma. BigQuery soporta exportaciones hacia Cloud Storage mediante `EXPORT DATA`, además de integraciones con herramientas de visualización, notebooks y conectores externos [11].

Un ejemplo simple de exportación tabular es el siguiente:

Listing 7: Exportación de resultados a Cloud Storage

```
EXPORT DATA OPTIONS(
  uri='gs://mi-bucket/reportes/ventas_*.csv',
  format='CSV',
  overwrite=true,
  header=true
) AS
SELECT *
FROM `proyecto.mart.ventas_mensuales`;
```

9.2. Exportación de modelos

Los modelos creados en BigQuery ML también pueden exportarse para ser reutilizados en otros entornos, por ejemplo cuando se desea integrar inferencia en otro flujo o profundizar el análisis en Python [10], [20].

Listing 8: Exportación de un modelo BigQuery ML

```
EXPORT MODEL `proyecto.ml.modelo_churn`
OPTIONS(
  uri='gs://mi-bucket/modelos/modelo_churn/'
);
```

9.3. Herramientas de consumo habituales

En la práctica, los resultados de BigQuery suelen consumirse desde:

- Looker o Looker Studio para visualización;
- notebooks de Python para análisis complementario;

- procesos ETL/ELT posteriores;
- aplicaciones que leen tablas, vistas o artefactos exportados;
- flujos de ciencia de datos que continúan fuera de BigQuery.

La idea clave es que BigQuery puede ser tanto un punto de llegada de datos como un punto de salida de productos analíticos.

10. Caso práctico integrador

Para articular los subtemas de la sesión, considérese el siguiente caso: una empresa de comercio electrónico almacena ventas, clientes y eventos de navegación en BigQuery. El objetivo es construir un flujo gobernado que permita analizar ventas, restringir acceso a información sensible y entrenar un modelo de predicción de abandono.

10.1. Diseño propuesto

C1. Cargar datos crudos en un *dataset* de *landing*.

C2. Transformarlos hacia un *dataset* `core` con tablas limpias y tipadas.

C3. Aplicar *policy tags* a columnas como correo, teléfono o identificador fiscal.

C4. Crear vistas autorizadas para usuarios de negocio.

C5. Construir una vista materializada con indicadores de ventas por día y región.

C6. Entrenar un modelo de abandono con BigQuery ML.

C7. Exportar resultados agregados a una herramienta externa para visualización.

10.2. Consulta agregada de soporte al modelo

Listing 9: Construcción de una tabla analítica para ML

```
CREATE OR REPLACE TABLE `proyecto.mart.clientes_entrenamiento` AS
SELECT
  c.cliente_id,
  c.antiguedad_meses,
  AVG(v.monto) AS monto_mensual,
  COUNTIF(s.tipo_ticket = 'reclamo') AS incidencias_soporte,
  AVG(e.sesiones_mes) AS consumo_promedio,
  c.abandono
FROM `proyecto.core.clientes` c
LEFT JOIN `proyecto.core.ventas` v
  ON c.cliente_id = v.cliente_id
LEFT JOIN `proyecto.core.soporte` s
  ON c.cliente_id = s.cliente_id
LEFT JOIN `proyecto.core.eventos_resumen` e
  ON c.cliente_id = e.cliente_id
```

```
16 GROUP BY
17   c.cliente_id,
18   c.antiguedad_meses,
19   c.abandono;
```

En este ejemplo convergen varios principios de la sesión: optimización razonable del SQL, construcción de una capa analítica reutilizable, separación de datos sensibles y uso posterior para un modelo predictivo.

11. Conclusiones

BigQuery representa una evolución importante del análisis de datos en la nube porque integra escalabilidad, administración simplificada, seguridad granular y capacidades analíticas avanzadas en una sola plataforma. Su diseño *serverless* no elimina la necesidad de disciplina técnica; al contrario, exige comprender cómo se factura el cómputo, cómo se gobiernan los accesos y cómo se diseñan consultas eficientes.

Desde la perspectiva de gobierno de datos, la plataforma ofrece mecanismos suficientes para implementar políticas serias de control: roles de IAM, separación por *datasets*, vistas autorizadas, seguridad por fila y por columna, y trazabilidad de eventos. Estas capacidades permiten publicar información con mayor seguridad y con mejor alineación al principio de mínimo privilegio.

Finalmente, BigQuery ML amplía el alcance del análisis al permitir que el estudiante construya modelos predictivos directamente con SQL. Esto fortalece la innovación aplicada, pero también demanda pensamiento crítico para interpretar resultados y rigor técnico para validar tanto el *pipeline* como el modelo. En conjunto, la sesión 9 muestra que el valor real de una plataforma analítica en la nube no está únicamente en consultar grandes volúmenes de datos, sino en hacerlo con gobierno, eficiencia y responsabilidad.

Referencias

[1] Google Cloud. «Audit policy tags,» visitado 7 de abr. de 2026. dirección: https://cloud.google.com/bigquery/docs/auditing-policy-tags

[2] Google Cloud. «Authorized datasets,» visitado 7 de abr. de 2026. dirección: https://docs.cloud.google.com/bigquery/docs/authorized-datasets

[3] Google Cloud. «Authorized views,» visitado 7 de abr. de 2026. dirección: https://cloud.google.com/bigquery/docs/authorized-views

[4] Google Cloud. «Basic roles and permissions,» visitado 7 de abr. de 2026. dirección: https://docs.cloud.google.com/bigquery/docs/access-control-basic-roles

[5] Google Cloud. «Best practices for using policy tags in BigQuery,» visitado 7 de abr. de 2026. dirección: https://docs.cloud.google.com/bigquery/docs/best-practices-policy-tags

[6] Google Cloud. «BigQuery IAM roles and permissions,» visitado 7 de abr. de 2026. dirección: https://docs.cloud.google.com/bigquery/docs/access-control

[7] Google Cloud. «BigQuery ML introduction,» visitado 7 de abr. de 2026. dirección: https://cloud.google.com/bigquery/docs/bqml-introduction

[8] Google Cloud. «BigQuery overview,» visitado 7 de abr. de 2026. dirección: https://docs.cloud.google.com/bigquery/docs/introduction

[9] Google Cloud. «BigQuery pricing,» visitado 7 de abr. de 2026. dirección: https://cloud.google.com/bigquery/pricing.html

[10] Google Cloud. «Export models,» visitado 7 de abr. de 2026. dirección: https://cloud.google.com/bigquery/docs/exporting-models

[11] Google Cloud. «Export table data,» visitado 7 de abr. de 2026. dirección: https://cloud.google.com/bigquery/docs/export-file

[12] Google Cloud. «Introduction to BigQuery row-level security,» visitado 7 de abr. de 2026. dirección: https://docs.cloud.google.com/bigquery/docs/row-level-security-intro

[13] Google Cloud. «Introduction to data governance in BigQuery,» visitado 7 de abr. de 2026. dirección: https://cloud.google.com/bigquery/docs/data-governance

[14] Google Cloud. «Introduction to materialized views,» visitado 7 de abr. de 2026. dirección: https://docs.cloud.google.com/bigquery/docs/materialized-views-intro

[15] Google Cloud. «Introduction to optimizing query performance,» visitado 7 de abr. de 2026. dirección: https://cloud.google.com/bigquery/docs/best-practices-performance-overview

[16] Google Cloud. «Optimize query computation,» visitado 7 de abr. de 2026. dirección: https://cloud.google.com/bigquery/docs/best-practices-performance-compute

[17] Google Cloud. «Overview of BigQuery storage,» visitado 7 de abr. de 2026. dirección: https://docs.cloud.google.com/bigquery/docs/storage_overview

[18] Google Cloud. «Restrict access with column-level access control,» visitado 7 de abr. de 2026. dirección: https://docs.cloud.google.com/bigquery/docs/column-level-security

[19] Google Cloud. «The CREATE MODEL statement,» visitado 7 de abr. de 2026. dirección: https://cloud.google.com/bigquery/docs/reference/standard-sql/bigqueryml-syntax-create

[20] Google Cloud. «The EXPORT MODEL statement,» visitado 7 de abr. de 2026. dirección: https://docs.cloud.google.com/bigquery/docs/reference/standard-sql/bigqueryml-syntax-export-model

[21] Google Cloud. «Understand BigQuery editions,» visitado 7 de abr. de 2026. dirección: https://docs.cloud.google.com/bigquery/docs/editions-intro